

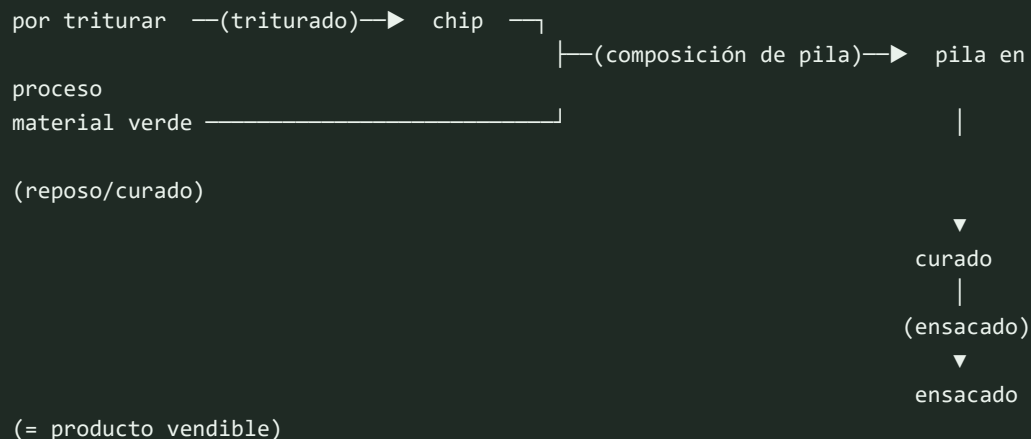
Spec — Módulo 5: Inventario, pilas y procesos ★ NÚCLEO

Casos de uso: CU-34 a CU-44 · **RF:** RF-24 a RF-30 **App backend:** nueva app `inventario`
· **Controladores:** `C_Inventario`, `C_Pilas`, `C_ProcesosPila` **Comportamiento detallado:**
`docs/referencia/Casos de uso extendidos.pdf` · **Datos:** `docs/referencia/Modelo de datos.pdf` §11.3 (Módulo 5)

Este módulo es el **corazón productivo** del sistema y el **camino crítico** del incremento: M6 (mezcla) y M7 (proyecciones) cuelgan de él. El concepto central es que el **inventario es un estado calculado**: nunca se teclea a mano, se recalcula solo cada vez que ocurre un movimiento (CU-44).

1. El modelo mental del inventario

El material atraviesa **etapas físicas** dentro de la planta. El `Inventario` guarda, por cada `Material` y cada etapa, cuánto hay:



Cada flecha es un **movimiento** que descuenta de una etapa y suma a otra. **Ningún movimiento deja cantidades en negativo**: si no hay stock suficiente en el origen, el movimiento se rechaza (CU-44, Excepción 2).

2. Modelos (Django) — `inventario/models.py`

Contrato de `docs/referencia/Modelo de datos.pdf` §11.3. `ProcesoPila` es de la **capa offline** (hereda `SincronizableModel` de `common`, igual que la recepción del Incremento 1: `id_local` UUID + `estado_sincronizacion`); el resto es online.

- **Pila**(`codigo` único, `fecha_inicio`, `estado`, `observaciones`). `estado` ∈ `en formacion` → `en proceso` → `en reposo` → `cerrada`. El `codigo` es correlativo autogenerated.
- **ComposicionPila**(`pila`, `material`, `volumen_m3`). Qué material y cuánto entró a una pila. Si se agrega el mismo material dos veces, se **suma** al registro existente, no se duplica (CU-36, Excepción 2).
- **ProcesoPila**(`id_local`, `pila` (NULL en triturado), `operador`, `tipo`, `fecha`, `temperatura`, `humedad`, `volumen_m3`, `estado_sincronizacion`). `tipo` ∈ `triturado`, `volteo`, `riego`, `harneado`, `reposo`, `ensacado`. Los campos de medición son opcionales según el tipo (temperatura en volteo, humedad en riego, volumen en triturado/ensacado). **pila es NULL para el triturado** (ocurre antes de armar la pila).
- **Inventario**(`material`, `etapa`, `volumen_m3`, `actualizado`) — **UNIQUE(material, etapa)**. `etapa` ∈ `por triturar`, `chip`, `pila en proceso`, `curado`, `ensacado`. Una fila por combinación material+etapa; se crea con cantidad 0 si no existe cuando se necesita.

3. Máquina de estados de la pila

Estado	Cómo se llega	CU
<code>en formacion</code>	al crear la pila	CU-35
<code>en proceso</code>	al confirmar la composición inicial (≥ 1 material)	CU-36
<code>en reposo</code>	al iniciar el reposo (requiere un <code>harneado</code> previo)	CU-41
<code>cerrada</code>	al ensacar y agotar el material curado de la pila	CU-42

Reglas de guarda: - **CU-40 (harneado)** solo si la pila está `en proceso`. - **CU-41 (reposo)** solo si la pila está `en proceso` **y tiene al menos un harneado registrado**. Al iniciar reposo se traslada en inventario `pila en proceso` → `curado`. - **CU-42 (ensacado)** solo si la pila está `en reposo`; traslada `curado` → `ensacado`. Cierra la pila (`cerrada`) solo cuando se agota el material curado; si queda, la pila sigue `en reposo`. - Volteo (CU-38) y riego (CU-39) se registran cuantas veces haga falta mientras la pila está `en proceso`; no cambian su estado, solo agregan a su historial.

4. Los movimientos de inventario (CU-44) — el punto más delicado

CU-44 no es una pantalla; es lógica de servidor que se dispara automáticamente cada vez que se crea un `ComposicionPila` o un `ProcesoPila` que traslada material. Implementarlo como un **servicio único** (p. ej. `inventario/services.py::aplicar_movimiento(...)`) que todos los CU de proceso llaman, dentro de una `transaction.atomic()`, para que el movimiento y la actualización de inventario sean una sola operación:

Disparador	CU	Movimiento en inventario
Triturado	CU-37	<code>por triturar</code> -volumen; <code>chip</code> +chip derivado
Composición de pila	CU-36	<code>chip</code> / material verde -cantidad; <code>pila en</code> <code>proceso</code> +cantidad
Inicio de reposo	CU-41	<code>pila en proceso</code> -cantidad de esa pila; <code>curado</code> +cantidad
Ensacado	CU-42	<code>curado</code> -cantidad; <code>ensacado</code> +cantidad

Reglas duras del servicio: 1. **Nunca negativo.** Si el origen no tiene stock suficiente, no se aplica el movimiento: se rechaza la operación con un error claro que indique el máximo disponible (CU-44 Excepción 2; también CU-36/37/42 Excepción de cantidad). 2. **Derivación en el triturado (CU-37):** el chip resultante se deriva con la densidad y el factor de reducción del `Material` (misma lógica de conversión del Incremento 1, `Material.derivar_chip`). Si el material no tiene factor configurado, se registra el triturado y se descuenta `por triturar`, pero el chip queda **pendiente de conversión** (no se inventa un valor) — CU-37 Excepción 3. 3. **Idempotencia offline:** como `ProcesoPila` viaja por la cola de sincronización, el servicio debe ser idempotente por `id_local` igual que la recepción del Incremento 1 (no aplicar dos veces el mismo movimiento si el lote se reenvía).

5. API

App `inventario`: serializers, viewsets, urls. Un `ViewSet` por controlador de `docs/referencia/Vistas y controladores.pdf`:

- `C_Inventario` → `/api/v1/inventario/` (**solo lectura**; agrupa por material y etapa, con búsqueda/filtro por material y por etapa — CU-34).
- `C_Pilas` → `/api/v1/pilas/` (crear pila CU-35; composición CU-36; ficha de trazabilidad interna CU-43 como detalle o acción anidada que devuelve composición + línea de tiempo de procesos).
- `C_ProcesosPila` → `/api/v1/procesos-pila/` (registrar triturado/volteo/riego/harneado/reposo/ensacado — CU-37 a 42; cada creación dispara el servicio de inventario de §4).
- **Permisos:** crear pilas y registrar procesos = `Operador` (y `Administrador`). Consultar inventario y trazabilidad = `Operador` y `Administrador`. Auditar los movimientos que trasladan material.
- **Sincronización:** los `ProcesoPila` capturados sin conexión entran por la misma cola offline del Incremento 1; extender el endpoint de sincronización para aceptar el tipo `proceso_pila` (o su equivalente), reusando la idempotencia por `id_local` ya probada.

6. Vistas (cablear el mockup existente)

Mockups en `frontend/src/pages/operacion/`: `Inventario.tsx`, `ListaPilas.tsx` ya existen como mock.

Vista (docs/referencia/Vistas y controladores.pdf)	CU	Qué hace
<code>V_Inventario</code>	CU-34	Existencias por material y etapa, con filtro
<code>V_ListaPilas</code> / <code>V_FormularioPila</code>	CU-35	Listado y alta de pilas
<code>V_ComposicionPila</code>	CU-36	Agregar material (del inventario disponible) a la pila
<code>V_RegistroProceso</code>	CU-37 a 42	Un formulario que registra el proceso según su tipo
<code>V_TrazabilidadPila</code>	CU-43	Ficha de la pila: composición + línea de tiempo de procesos

7. Criterios de aceptación

1. **Ciclo completo de una pila:** crear pila → componer con chip + verde → volteo → riego → harneado → reposo → ensacado → pila `cerrada` . El inventario refleja cada traslado y **nunca queda negativo**.
2. **Triturado deriva chip** desde el volumen y los parámetros del material; sin factor configurado, deja el chip pendiente sin inventar valor.
3. **Composición que excede el stock** disponible se rechaza indicando el máximo.
4. **Reposo bloqueado** si la pila no tiene un harneado previo.
5. **CU-44 automático:** consultar el inventario tras cada proceso muestra las cantidades correctas sin intervención manual.
6. **Idempotencia:** reenviar un `ProcesoPila` ya sincronizado no aplica el movimiento dos veces.

8. Documentación de diseño

Secuencias de diseño (normal + excepción) para **CU-34 a CU-44** (11 CU). CU-44 se diagrama como la reacción del `:Inventario` disparada por `C_ProcesosPila` .